Virtual League

The Virtual League mirrors the Main League, with the sole exception being that the robots used are designed by Student Robotics, not individual teams. Everyone is on an even playing field with regards to the hardware of their robot – it is the strength of the software solution that counts.

To gain points, robots must collect pallets of their own colour and deposit them in the scoring districts (small white squares). The full scoring rules can be found here.

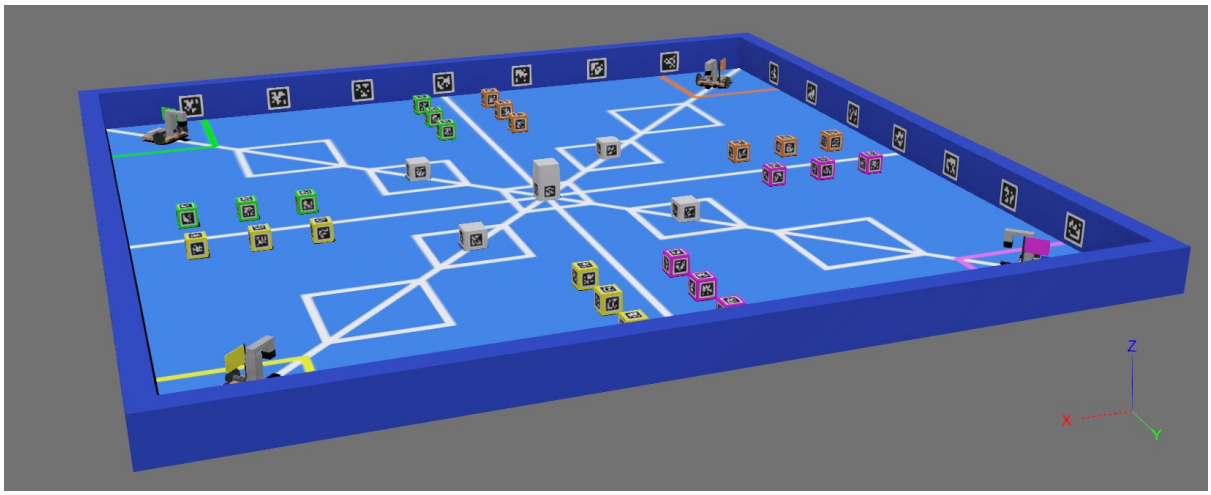

Figure 1: Screenshot of the virtual arena captured using Webots R2023b. The robot and pallets belonging to a zone are colour coded according to that zone's colour. Fiducial markers are positioned around the arena, on the pallets, and on the high-rises to enable robots to position themselves accurately using the Vision API.

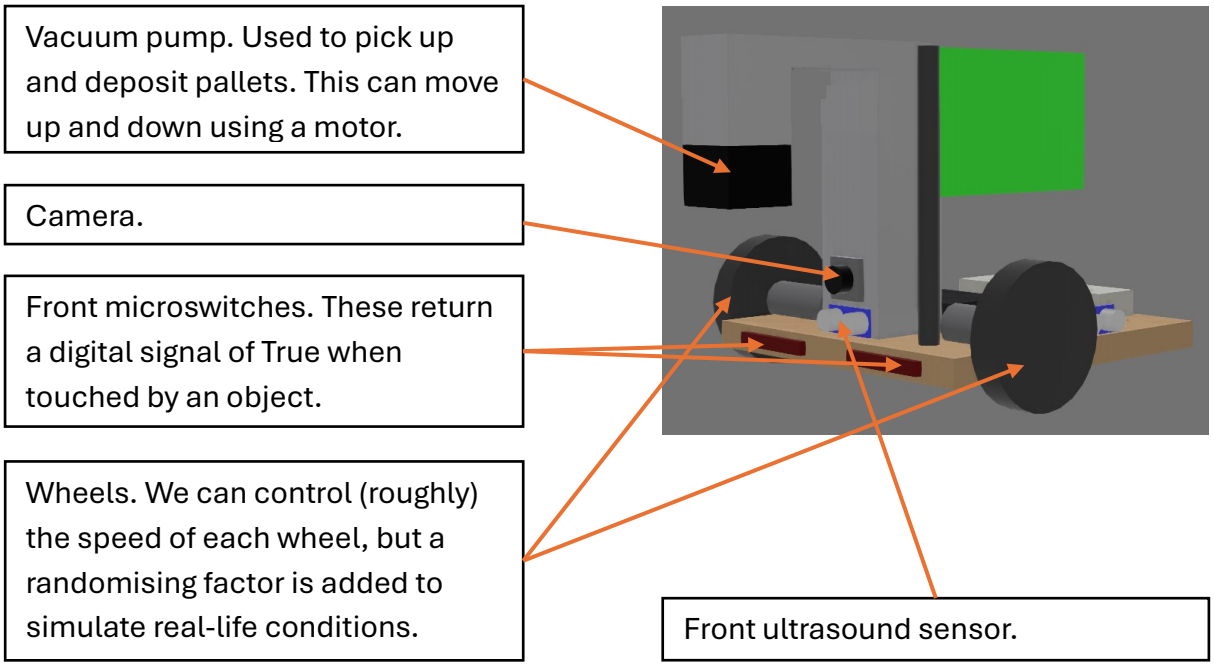

Figure 2: Screenshot of the robot that all teams used, complete with annotations. Only components that we used are annotated.

Navigating the virtual arena – a high-level overview

Using the default `sr.robot3` library, provided by Student Robotics, we could interface with the camera of the virtual robot. The `camera.see()` command, part of the Vision API provided in this library, takes a picture using the camera and returns a list of Marker objects, with each Marker object representing a fiducial marker (see **Figure 3** below). Each pallet has a unique ID in a number range corresponding to its zone.

Item	Marker Numbers	Marker Size (mm)
Arena boundary	0 - 27	150
Zone 0 pallets	100-119	80
Zone 1 pallets	120-139	80
Zone 2 pallets	140-159	80
Zone 3 pallets	160-179	80
Outer high-rises	Near zone 0: 195 Near zone 1: 196 Near zone 2: 197 Near zone 3: 198	80
Central high-rise	199	80

Table 1: Reproduced from Rules 2025 — Urban Heights | Student Robotics.

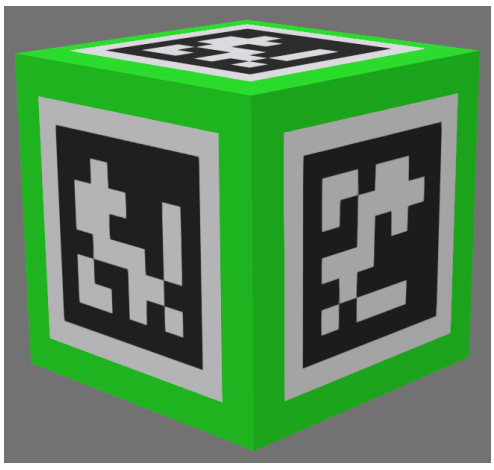

Figure 3: Green-coloured pallet with 3 fiducial markers visible. These markers have the same ID, but different orientations. Each marker is represented as a single Python object, regenerated upon each refresh of the camera.

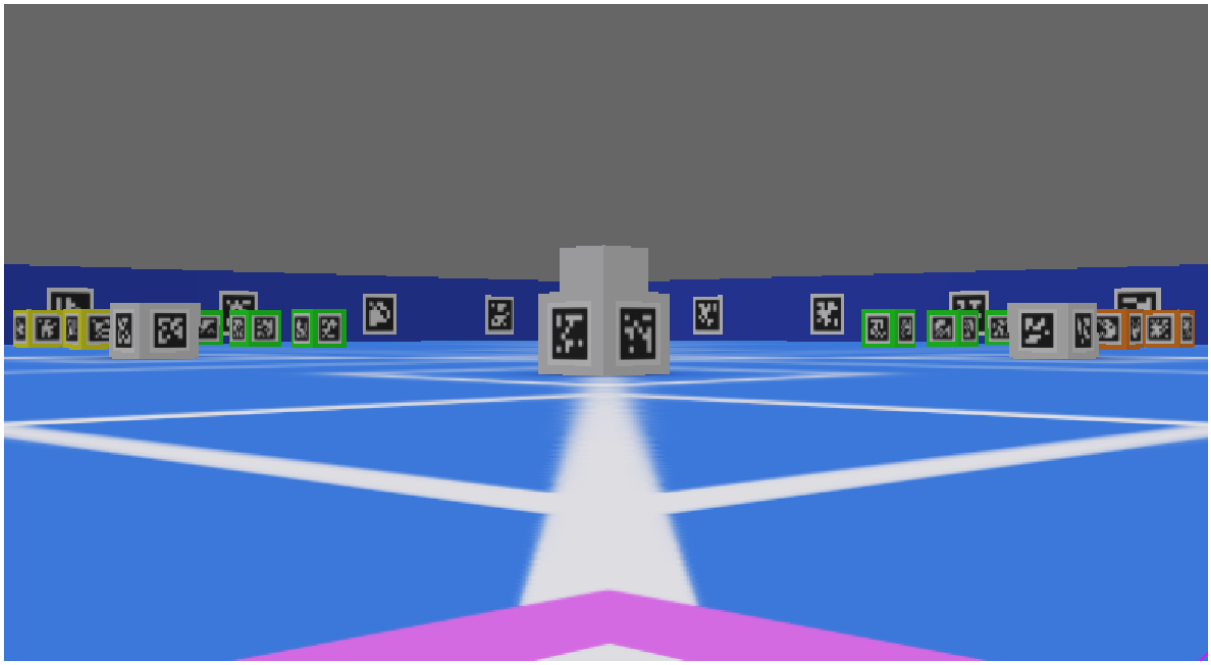

Figure 4: Camera POV from the default position of the magenta robot. No magenta pallets are visible, so the robot must rotate to locate them. To gain points, the robot must pick up pallets of its own colour, and deposit them in any of the 9 scoring districts.

Once started, our robot was programmed to turn clockwise on the spot until it could see a Marker object of its own colour.

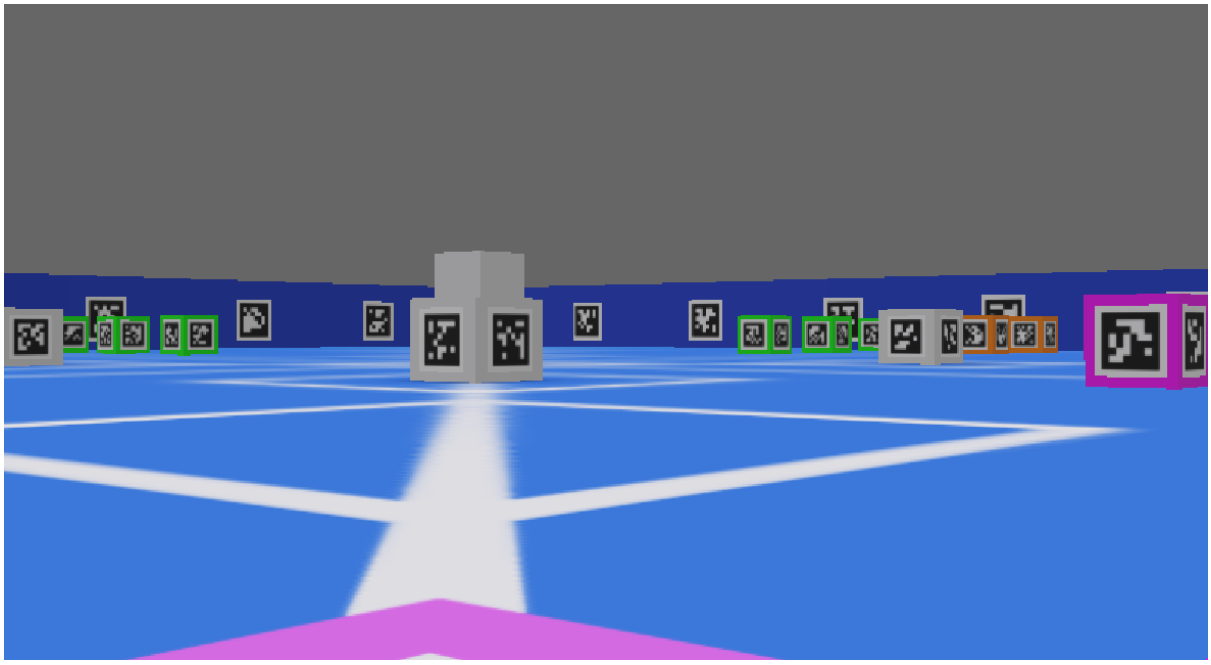

Figure 5: Here, the robot has stopped as a magenta pallet is now in view. As there are 2 magenta markers in view, the robot must now decide which of these it wants to target.

In order for the vacuum pump to reliably pick up a pallet, the robot must face the pallet square-on. It is therefore preferable to pick a face with a lower absolute yaw, as this

means that less time is wasted rotating the robot. As such, the face picked here would be the left most visible face on the magenta pallet.

The ideal path to this face is shown below in **Figure 6**.

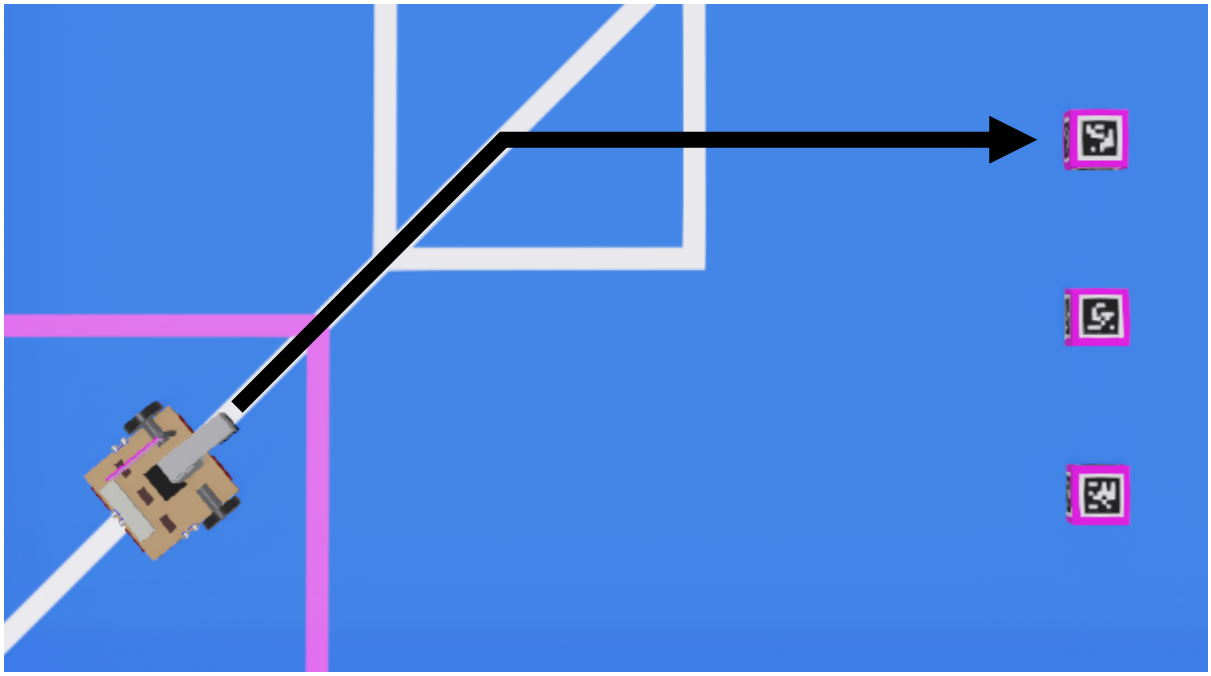

Figure 6: Top-down view of the arena showing the ideal path (in black) to the chosen pallet face.

Trigonometric calculations can now be performed to determine how far the pallet must travel along the white line (“travel distance”) before turning to locate the pallet.

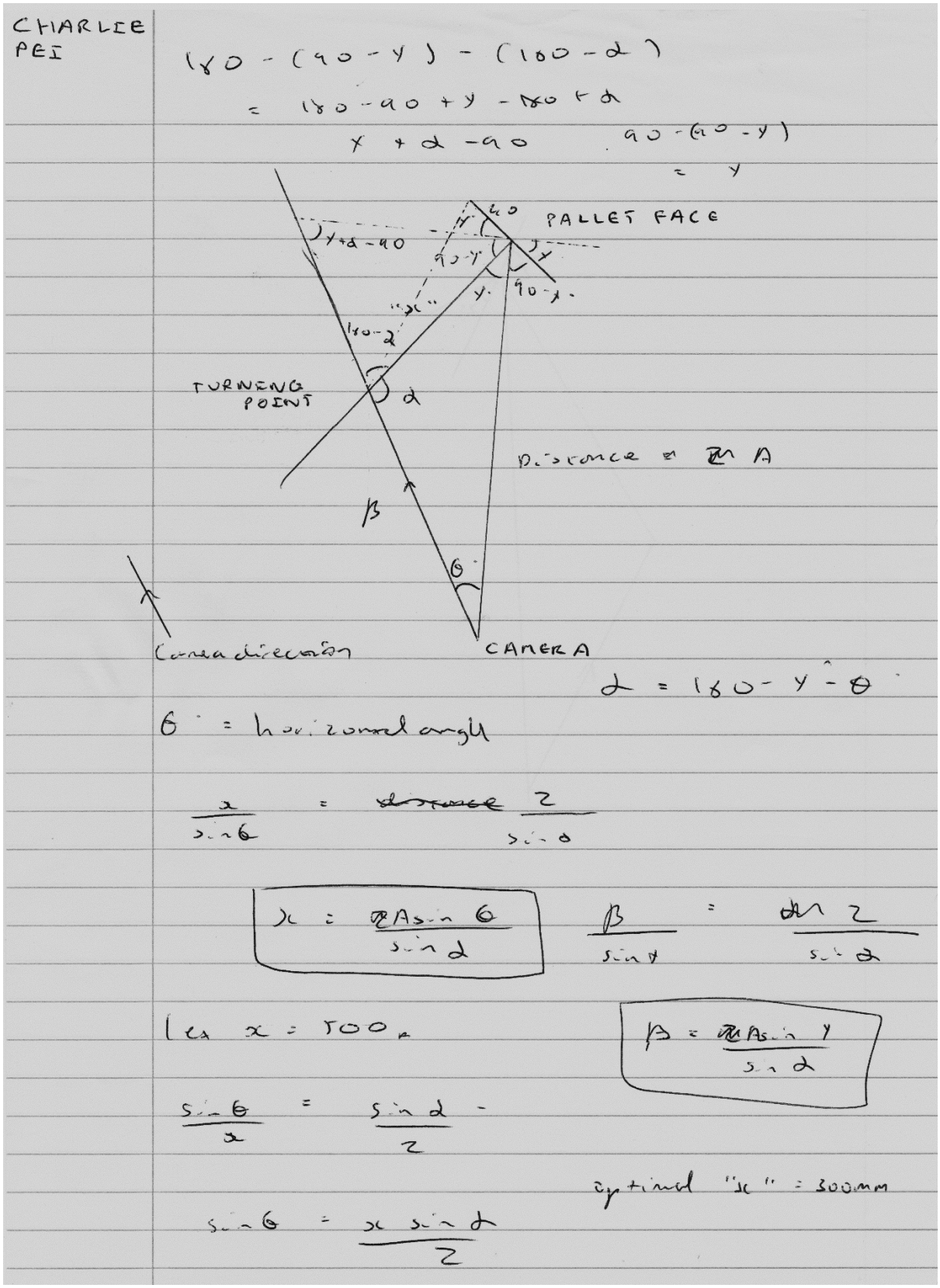

Figure 7: Original calculations sheet showing how the travel distance is calculated.

There is no easy way to program the robot to travel a set distance as this facility is not provided in the default `sr.robot3` library. It was therefore decided to calculate the time it would take for the robot to travel the required distance at a certain speed, and to

stop powering the wheels' motors after this point, enabling the robot to travel the set distance.

In essence:

$$time\ (s) = \frac{distance\ (m)}{speed\ (ms^{-1})}$$

Having obtained the distance required via trigonometric calculations, we must now deduce the speed of the robot. The power provided to each wheel's motor can be set to a float between -1 and 1 inclusive, but to convert this value to a speed with units ms^{-1}, we need to also know the radius of each wheel, as well as the motor's maximum angular speed. All this information can be found in the `MotoAssembly.proto` file used to setup the robot, provided by Student Robotics.

Thus, the following equation can be derived:

$$linear\ speed\ (ms^{-1}) = motor\ power * maximum\ angular\ speed\ (rad\ s^{-1}) * radius\ (m)$$

We then multiply this linear speed by 1.1 to account for the random variations in motor speed, before substituting it back into the first equation to obtain the total time required to travel a set distance at a certain motor power.

We can now obtain the current system time using `robot.time()`, save it to a variable, and simultaneously the set the power of both motors to the desired value. We decided to use 0.5 as this provided the best comprise between speed and reliability – the greater the power provided, the more likely it is for the robot to go off-course, as the random variations in speed differ for each wheel.

After the calculated time has elapsed, both motors can be simultaneously stopped. The robot can now turn on the spot to align perfectly with the target fiducial marker and move in a straight line towards it. Once the front microswitches and ultrasound sensor detect that the robot is touching the pallet, it can be stopped, and the vacuum pump can be activated to pick up the pallet.

A similar process is repeated for depositing the pallet in one of the many districts around the arena.

Running the simulation – how-to

Should you wish, you can actually run the code we submitted for the Virtual League yourself. Please note that this code is different to the code used for the Main League.

Requirements

Software	Download link
Python 3.11	https://www.python.org/downloads/
Webots R2023b	https://github.com/cyberbotics/webots/releases/tag/R2023b
sbot_simulator-2025.1.1	https://github.com/srobo/sbot_simulator/releases/tag/2025.1.1

Table 2: Requirements list for running the code.

Installation instructions are available on our GitHub repository here: https://github.com/octa-2049/student-robotics.

Video demonstration

You can watch our robot compete against others in the official competition video here: https://youtu.be/p0KxrRNTGBs.

Use the video chapters in the video description to find our matches or use the table below. We played in matches 1, 8, 15, and 20. Please note that matches are numbered from 0 onwards, not 1. Our team code is **KEG**.

Match no.	Link	Result
1	https://youtu.be/p0KxrRNTGBs&t=891s	1st – 8 league points earned
8	https://youtu.be/p0KxrRNTGBs&t=2781s	1st – 8 league points earned
15	https://youtu.be/p0KxrRNTGBs&t=4671s	1st – 8 league points earned
20	https://youtu.be/p0KxrRNTGBs&t=6021s	1st – 8 league points earned

Table 3: Match list with video links. We were the only team to come 1st in all the Virtual League matches we played.